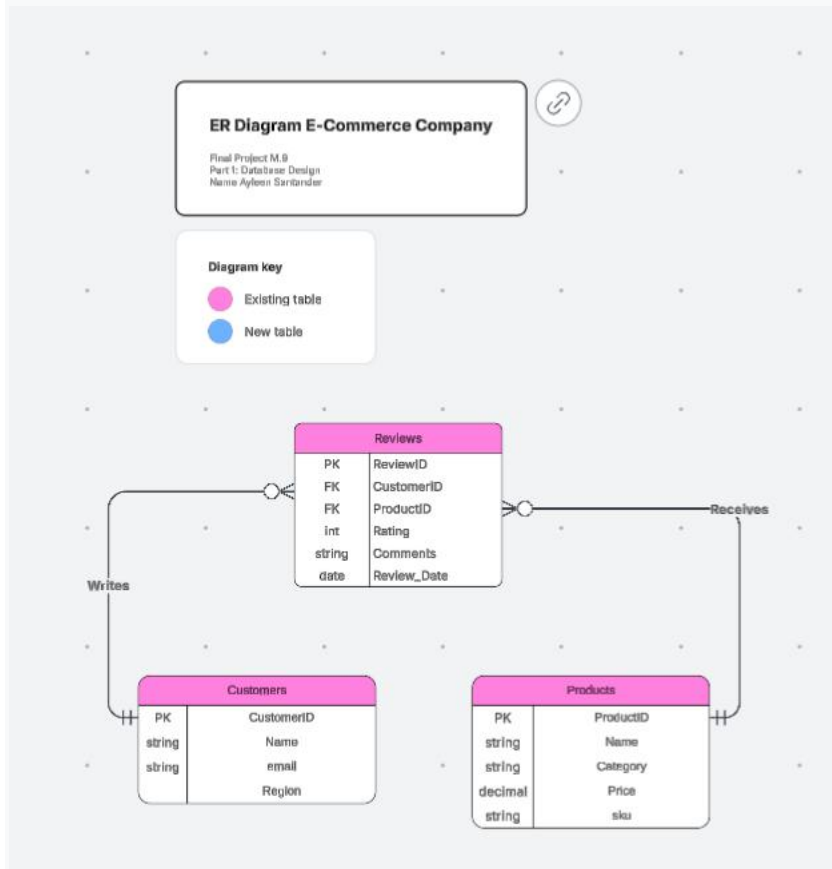
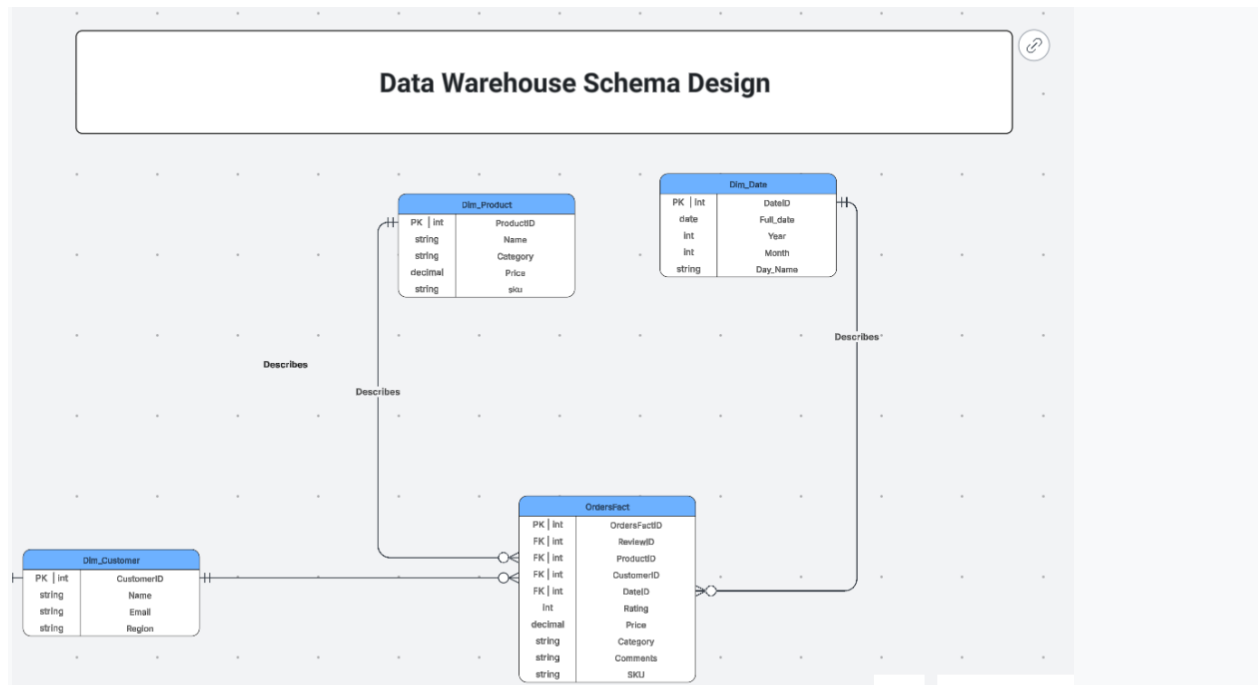


Final Project Part 1

Part 1: Database Design

1. Design an ER diagram covering at minimum:
 - Customers: name, email, region.
 - Products: name, category, price, and the SKU used by the source systems.
 - Reviews: customer, product, rating, comment, date.
2. Normalize to third normal form. Two dependencies worth checking: a customer's region determines their country, and a product's category repeats across rows—state which normal form your tables reach.





- Define primary keys, foreign keys with explicit **ON DELETE** actions, and **CHECK** constraints on bounded columns such as the rating.

Decide whether the review-to-customer link may be **NULL**. Some reviews in the data cannot be matched to a known customer. Justify whichever way you go.

```

1  /*I use NOT NULL because the business rule requires every customer to write a comment in order to
2  finalize their review, this comment is exactly what triggers the points award. Since the system doesn't
3  allow the customer to complete the process without submitting a comment, NULL should never be a valid state
4  for this column. Enforcing NOT NULL at the database level guarantees this rule holds even if the application's
   validation fails, has a bug, or a review gets inserted through another route.*/
  
```

- Add a constraint that makes re-running an import safe. The three feeds issue their own record IDs, and those IDs can collide across sources.

```

24
25 • CREATE TABLE reviews (
26     reviewid      INT AUTO_INCREMENT PRIMARY KEY,
27     customerid    INT          NOT NULL,
28     productid     INT          NOT NULL,
29     rating        INT          NOT NULL CHECK (rating BETWEEN 1 AND 5),
30     comments      TEXT,
31     review_date   DATE          NOT NULL DEFAULT (CURRENT_DATE),
32     CONSTRAINT fk_reviews_customer FOREIGN KEY (customerid) REFERENCES customers(customerid)
33         ON DELETE CASCADE
34         ON UPDATE CASCADE,
35     CONSTRAINT fk_reviews_product FOREIGN KEY (productid) REFERENCES products(productid)
36         ON DELETE CASCADE
37         ON UPDATE CASCADE,
38     CONSTRAINT uq_customer_product_review UNIQUE (customerid, productid)
39 );

```

5. Implement it in MySQL as a single script that drops and recreates the database from scratch.

```

1  /*Final Project Part 1 & 2
2   Name Ayleen Santander*/
3   /*Part 1: Database Design*/
4 • CREATE DATABASE IF NOT EXISTS E_CommerceCompanyDB;
5 • USE E_CommerceCompanyDB;
6
7 • CREATE TABLE customers (
8     customerid    INT AUTO_INCREMENT PRIMARY KEY,
9     name          VARCHAR(100) NOT NULL,
10    email         VARCHAR(150) NOT NULL,
11    region        VARCHAR(100) NOT NULL,
12    CONSTRAINT uq_customers_email UNIQUE (email)
13 );
13 );
14 • CREATE TABLE products (
15     productid     INT AUTO_INCREMENT PRIMARY KEY,
16     name          VARCHAR(150) NOT NULL,
17     category      VARCHAR(100) NOT NULL,
18     price         DECIMAL(10,2) NOT NULL CHECK (price >= 0)
19 );
20 /** ADD THE NEW COLUMN SKU*/
21 • ALTER TABLE products ADD COLUMN SKU VARCHAR(255) NOT NULL;
22 /*Check the new column was added*/
23 • SELECT * FROM products;

```

```

24
25 • CREATE TABLE reviews (
26     reviewid      INT AUTO_INCREMENT PRIMARY KEY,
27     customerid    INT          NOT NULL,
28     productid     INT          NOT NULL,
29     rating        INT          NOT NULL CHECK (rating BETWEEN 1 AND 5),
30     comments      TEXT,
31     review_date   DATE          NOT NULL DEFAULT (CURRENT_DATE),
32     CONSTRAINT fk_reviews_customer FOREIGN KEY (customerid) REFERENCES customers(customerid)

```

```

32     CONSTRAINT fk_reviews_customer FOREIGN KEY (customerid) REFERENCES customers(customerid)
33         ON DELETE CASCADE
34         ON UPDATE CASCADE,
35     CONSTRAINT fk_reviews_product FOREIGN KEY (productid) REFERENCES products(productid)
36         ON DELETE CASCADE
37         ON UPDATE CASCADE,
38     CONSTRAINT uq_customer_product_review UNIQUE (customerid, productid)
39 );
40 /*I use NOT NULL because the business rule requires every customer to write a comment in order to
41 finalize their review, this comment is exactly what triggers the points award. Since the system doesn't

```

```

1  //
2  /*I use NOT NULL because the business rule requires every customer to write a comment in order to
3  finalize their review, this comment is exactly what triggers the points award. Since the system doesn't
4  allow the customer to complete the process without submitting a comment, NULL should never be a valid state
5  for this column. Enforcing NOT NULL at the database level guarantees this rule holds even if the application's
6  validation fails, has a bug, or a review gets inserted through another route.*/

```

```

99     /*Create the Dim Date table*/
100 • CREATE TABLE Dim_Date (
101     DateID INT PRIMARY KEY,
102     OrderDate DATE,
103     DayOfWeek VARCHAR(10),
104     MonthName VARCHAR(10),
105     Quarter INT,
106     Year INT
107 ) ENGINE=InnoDB;
108
109 • ALTER TABLE Dim_Date RENAME COLUMN OrderDate TO Full_Date;
110 • SELECT * FROM Dim Date;

```

```

120
127 • CREATE TABLE OrdersFact (
128     OrdersFactID INT AUTO_INCREMENT PRIMARY KEY,
129     ReviewID INT,
130     CustomerID INT,
131     ProductID INT,
132     DateID INT,
133     Rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
134     Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
135     Category VARCHAR(100) NOT NULL,
136     Comments TEXT NOT NULL,
137     SKU VARCHAR(255) NOT NULL,
138
139     FOREIGN KEY (ReviewID) REFERENCES Reviews(ReviewID),
140     FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
141     FOREIGN KEY (ProductID) REFERENCES Products(ProductID),
142     FOREIGN KEY (DateID) REFERENCES Dim_Date(DateID)
143 ) ENGINE=InnoDB;
144
145
146
147
148
149
150
151 /*Load Data into the OrdersFact
152 Load data from reviews*/
153 • INSERT INTO OrdersFact
154     (ReviewID, CustomerID, ProductID, DateID,
155     Rating, Price, Category, Comments, SKU)
156 SELECT
157     r.ReviewID, r.CustomerID, r.ProductID, d.DateID,
158     r.Rating, p.Price, p.Category, r.Comments, p.SKU
159 FROM Reviews r
160 JOIN Products p
161     ON r.ProductID = p.ProductID
162 LEFT JOIN Customers c

```

OrdersFactID	ReviewID	CustomerID	ProductID	DateID	Rating	Price	Category	Comments	SKU
1	1	1	1	NULL	5	24.99	Electronics	Works great, very responsive.	ELE-0001
2	2	2	2	NULL	4	349.00	Furniture	Sturdy but assembly took a while.	FUR-0002
3	3	3	3	NULL	5	199.99	Electronics	Excellent sound quality.	ELE-0003
4	4	1	3	NULL	4	199.99	Electronics	Good value for the price.	ELE-0003
5	5	4	4	NULL	5	289.99	Furniture	Very comfortable for long work...	FUR-0004
6	6	5	5	NULL	4	89.99	Electronics	Great tactile feedback, a bit loud.	ELE-0005
7	7	6	6	NULL	3	34.50	Furniture	Decent brightness but flimsy ba...	FUR-0006

Part 2: Data Integration

1. **Stage first.** Create one staging table per source with **every column typed as text**, then validate on the way into your final tables. A rating of 6 aborts a load into a typed column;

in a text column it waits until your cleaning logic decides. Loading straight into final tables is the most common way this assignment goes wrong.

2. Load the CSV (`customer_survey.csv`) with `LOAD DATA LOCAL INFILE`.

```
220  /*Part 2: Data Integration
221  Stage first. Create one staging table per source with every column typed as text,
222  then validate on the way into your final tables. */
223  CREATE TABLE customer_survey (
224      response_id VARCHAR(20) PRIMARY KEY,
225      submitted_on VARCHAR(20),
226      respondent_name VARCHAR(255) NOT NULL,
227      respondent_email VARCHAR(100) NOT NULL,
228      product_label VARCHAR(255),
229      sku VARCHAR(20),
230      rating_1_5 TINYINT,
231      open_comment TEXT,
232      channel VARCHAR(50)
233  );
```

response_id	submitted_on	respondent_name	respondent_email	product_label	sku	rating_1_5	open_comment	channel
SV-2000	2024-03-26	Tessa Vos	tessa.vos@penpost.co	Nimbus Indoor Air Monitor (EL...	EL-2067	5	Replacement was sent out the s...	post-purchase
SV-2001	2024-10-14	Nadia Hassan	nadia.hassan@brightpost.net	Voltmark 20000mAh Power Ba...	EL-2048	3	Mixed. Works well, looks cheap.	post-purchase
SV-2002	2024-08-21	Nadia Hassan	nadia.hassan@brightpost.net	Kestrel Resistance Band Set (OF...	OF-4031	2	Plastic feels cheap and the finis...	post-purchase
SV-2003	2024-09-18	Mira Kaur	mira.kaur@corvid.io	Auralis A2 Wireless Earbuds	EL-2011	2	Drains overnight even when sw...	email survey
SV-2004	2025-01-09	Malik Boateng	malik.boateng@penpost.co	Meadowlark Ceramic Dinner Set	HK-1090	4	Does one thing and does it well.	email survey

3. Load the JSON (`web_feedback.json`). It is one object containing an array, so a line-by-line load will not work. Use `JSON_TABLE` or a Python parser. Watch for a field that is sometimes absent rather than empty, and a field whose type varies between records.

```
1  #2. JSON for web feedback data:
2  import pandas as pd
3  import json
4  # 1. Parse JSON data
5  with open('web_feedback.json', mode='r') as file:
6      data = json.load(file)
7
8  df = pd.DataFrame(data)
9  print(df)
10
11  print('--'*30)
12
```

```

19         2.1 ... {'submission_id': 'WF-5019', 'received_at': '2...
20         2.1 ... {'submission_id': 'WF-5020', 'received_at': '2...
21         2.1 ... {'submission_id': 'WF-5021', 'received_at': '2...
22         2.1 ... {'submission_id': 'WF-5022', 'received_at': '2...
23         2.1 ... {'submission_id': 'WF-5023', 'received_at': '2...
24         2.1 ... {'submission_id': 'WF-5024', 'received_at': '2...
25         2.1 ... {'submission_id': 'WF-5025', 'received_at': '2...
26         2.1 ... {'submission_id': 'WF-5026', 'received_at': '2...
27         2.1 ... {'submission_id': 'WF-5027', 'received_at': '2...
28         2.1 ... {'submission_id': 'WF-5028', 'received_at': '2...
29         2.1 ... {'submission_id': 'WF-5029', 'received_at': '2...
30         2.1 ... {'submission_id': 'WF-5030', 'received_at': '2...
31         2.1 ... {'submission_id': 'WF-5031', 'received_at': '2...
32         2.1 ... {'submission_id': 'WF-5032', 'received_at': '2...
33         2.1 ... {'submission_id': 'WF-5033', 'received_at': '2...
34         2.1 ... {'submission_id': 'WF-5034', 'received_at': '2...
35         2.1 ... {'submission_id': 'WF-5035', 'received_at': '2...
36         2.1 ... {'submission_id': 'WF-5036', 'received_at': '2...
37         2.1 ... {'submission_id': 'WF-5037', 'received_at': '2...

```

[38 rows x 4 columns]

#2. Display parsed data# Display parsed data

```

print(f"{'submission_id':<15} {'received_at':<25} {'customer_name':<20} {'customer_email':>35} "
      f"{'verified':<10} {'product_sku':<15} {'product_title':<40} {'score':<8} "
      f"{'message':<80} {'tags':<70}")
print('---' * 60)

for submission in data['submissions']:
    submission_id = submission['submission_id']
    received_at = submission['received_at']
    customer_name = submission['customer']['name']
    customer_email = submission['customer']['email'] or 'N/A'
    customer_verified_purchaser = submission['customer']['verified_purchaser']
    product_sku = submission['product']['sku']
    product_title = submission['product']['title']
    score = submission['score']
    message = submission['message']
    tags = submission.get('tags', [])
    tags_str = ', '.join(tags) if tags else 'none'

    print(f"{'submission_id':<15} {'received_at':<25} {'customer_name':<20} {'customer_email':>35} "
          f"{'str(customer_verified_purchaser):<10} {'product_sku':<15} {'product_title':<40} {'score':<8} "
          f"{'message':<80} {'tags_str':<70}")

print('---' * 60)

```

WF-5022	2024-01-20T14:03:00Z	Nia Adeyemi	nia.adeyemi@penpost
WF-5023	2024-03-15T12:12:00Z	Anya Sorokin	anya.sorokin@penpost
WF-5024	2024-03-21T16:07:00Z	Rosa Alvarez	rosa.alvarez@riverbox.
WF-5025	2025-06-24T06:01:00Z	Marcus Quinn	marcus.quinn@riverbox.
WF-5026	2024-05-04T20:23:00Z	Silas Lindgren	
WF-5027	2024-06-05T06:10:00Z	Lena Novak	lena.novak@mailhaven.
WF-5028	2025-02-17T13:49:00Z	Elif Demir	elif.demir@brightpost.
WF-5029	2025-02-01T20:16:00Z	Lena Novak	lena.novak@mailhaven.
WF-5030	2025-05-03T12:50:00Z	Lena Novak	lena.novak@mailhaven.
WF-5031	2025-06-12T14:15:00Z	Caleb Ruiz	caleb.ruiz@brightpost.
WF-5032	2025-03-30T09:52:00Z	Silas Lindgren	silas.lindgren@riverbox.
WF-5033	2024-04-11T16:11:00Z	Ines Ferreira	ines.ferreira@corvid
WF-5034	2024-07-08T18:21:00Z	Lena Novak	
WF-5035	2025-05-01T22:17:00Z	Emeka Nwosu	
WF-5036	2025-04-11T08:52:00Z	Keiko Mori	keiko.mori@mailhaven.
WF-5037	2024-11-17T19:34:00Z	Noor Rahimi	

Total records displayed: 38

['submission_id', 'received_at', 'score', 'message', 'tags', 'customer.name', 'customer.email',

```
# Normalize the submissions
df_survey = pd.json_normalize(data['submissions'])
print(df_survey.columns.tolist())

# Rename nested dot-notation columns to clean names
df_survey = df_survey.rename(columns={
    'customer.name': 'customer_name',
    'customer.email': 'customer_email',
    'customer.verified_purchaser': 'verified',
    'product.sku': 'product_sku',
    'product.title': 'product_title'
})

# Extract
norm_data = df_survey[['submission_id', 'received_at', 'customer_name', 'customer_email',
                        'verified', 'product_sku', 'product_title', 'score',
                        'message', 'tags']].copy()

print("Normalized data:")
print(norm_data.to_string())
print('--' * 30)
```


Normalized data:

	submission_id	received_at	customer_name	customer_email	verified
0	WF-5000	2024-10-29T21:48:00Z	Sofia Marchetti	sofia.marchetti@mailhaven.com	True
1	WF-5001	2024-12-23T16:23:00Z	Silas Lindgren	silas.lindgren@riverbox.com	True
2	WF-5002	2024-04-01T12:00:00Z	Clara Duarte	clara.duarte@corvid.io	True
3	WF-5003	2024-05-11T10:55:00Z	Clara Duarte	clara.duarte@corvid.io	True
4	WF-5004	2024-06-30T19:50:00Z	Devin Halloran	devin.halloran@brightpost.net	True
5	WF-5005	2024-01-16T07:22:00Z	Hiro Tanaka	hiro.tanaka@mailhaven.com	True
6	WF-5006	2024-12-03T16:05:00Z	Petra Kovac	petra.kovac@mailhaven.com	True
7	WF-5007	2024-11-21T14:11:00Z	Joel Abbott	NaN	False
8	WF-5008	2025-03-21T14:55:00Z	Nadia Hassan	nadia.hassan@brightpost.net	False
9	WF-5009	2025-02-16T12:52:00Z	Owen Bright	owen.bright@riverbox.com	False
10	WF-5010	2024-05-21T18:34:00Z	Petra Kovac	petra.kovac@mailhaven.com	True
11	WF-5011	2025-06-01T10:58:00Z	Tomas Vesely	tomas.vesely@riverbox.com	True
12	WF-5012	2024-08-15T20:17:00Z	Yara Haddad	NaN	False
13	WF-5013	2025-05-11T11:42:00Z	Tessa Vos	tessa.vos@penpost.co	True
14	WF-5014	2024-10-04T14:26:00Z	Silas Lindgren	silas.lindgren@riverbox.com	True
15	WF-5015	2025-05-18T18:17:00Z	Kwame Mensah	kwame.mensah@corvid.io	True
16	WF-5016	2025-05-21T16:13:00Z	Felix Brandt	felix.brandt@riverbox.com	True
17	WF-5017	2024-10-19T20:02:00Z	Rahul Sood	NaN	False
18	WF-5018	2024-12-28T15:29:00Z	Emeka Nwosu	emeka.nwosu@mailhaven.com	False
19	WF-5019	2025-02-24T16:15:00Z	Sanne de Vries	sanne.devries@brightpost.net	True

```
# Save to CSV
norm_data.to_csv( path_or_buf: 'web_feedback_from_json.csv', index=False)
print("Saved to web_feedback_from_json.csv")
print('--' * 30)
```

Saved to web_feedback_from_json.csv

web_feedback_from_json.csv

Parse the JSON file after fetch from Python

```
259  /*step 2: Create table for JSON after change format in python*/
260  • CREATE TABLE web_feedback_from_json (
261      submission_id text,
262      received_at text,
263      customer_name text,
264      customer_email text,
265      verified text,
266      product_sku text,
267      product_title text,
268      score text,
269      message TEXT
```

```

72 • SELECT * FROM web_feedback_from_json ;
73 /*Check the duplicates*/
74 • SELECT COUNT(*) AS num_rows FROM web_feedback_from_json;
75 /*Whom is duplicated*/
76 • SELECT customer_name, submission_id, COUNT(*) AS num_submissions
77 FROM web_feedback_from_json
78 GROUP BY customer_name, submission_id;

```

submission_id	received_at	customer_name	customer_email	verified	product_sku	product_title	score	message
WF-5000	2024-10-29T21:48:00Z	Sofia Marchetti	sofia.marchetti@mailhaven.com	True	OG-5041	Inkwell A5 Hardcover Notebook	2	Manual is a poor translati
WF-5001	2024-12-23T16:23:00Z	Silas Lindgren	silas.lindgren@riverbox.com	True	OG-5012	Latitude Standing Desk Converter	2	Capacity is nowhere near
WF-5002	2024-04-01T12:00:00Z	Clara Duarte	clara.duarte@corvid.io	True	EL-2011	Auralis A2 Wireless Earbuds	3	Battery life is fine but the
WF-5003	2024-05-11T10:55:00Z	Clara Duarte	clara.duarte@corvid.io	True	OF-4056	Summit Foam Roller	5	Came a day earlier than th
WF-5004	2024-06-30T19:50:00Z	Devin Halloran	devin.halloran@brightpost.net	True	OG-5023	Latitude Ergonomic Mesh Chair	4	Good value at the sale pri

4. **Load the XML** (`external_reviews.xml`). Try `LOAD XML INFILE` first, then explain in your report what it could and could not read from this file — the answer depends on where the data physically sits. A Python parser using `xml.etree.ElementTree` is acceptable if you can say why it was needed.

```

283 /*Step 3: Create a XML table*/
284 • CREATE TABLE external_reviews_native (
285     id          VARCHAR(20),
286     postedOn    VARCHAR(20),
287     displayName VARCHAR(100),
288     email       VARCHAR(255),
289     country     VARCHAR(10),
290     item        VARCHAR(255),
291     score       VARCHAR(10),
292     body        TEXT,
293     helpfulVotes INT

```

After multiple errors with the system mentioned I need to remove the security mode.

In this case I applied an extra step because I cannot fetch the data until I remove the private mode, I call the cmd in the shell and write the address into it.

```

296 /*Show variables inside*/
297 • SHOW VARIABLES LIKE 'secure_file_priv';
298 • LOAD XML INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/external_re
299 INTO TABLE external_reviews_native
300 ROWS IDENTIFIED BY '<review>';
301
302 /*check the load xml file*/

```

Variable_name	Value
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

```
C:\Windows\system32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>cmd
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>copy "C:\Users\aylee\Downloads\external_reviews.xml" "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\|"
```

And after that the system shows me 1 file copied.

```
C:\Windows\system32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>cmd
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>copy "C:\Users\aylee\Downloads\external_reviews.xml" "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\"
1 file(s) copied.

C:\Users\aylee>
```

And after that write again the below statement.

```
296 /*Show variables inside*/
297 • SHOW VARIABLES LIKE 'secure_file_priv';
298 • LOAD XML INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ext
299 INTO TABLE external_reviews_native
300 ROWS IDENTIFIED BY '<review>';
301
302 /*check the load xml file*/
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
Variable_name	Value			
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\			

```
C:\Windows\system32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>cmd
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>copy "C:\Users\aylee\Downloads\external_reviews.xml" "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\"
1 file(s) copied.

C:\Users\aylee>dir "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\|"
```

And the system recognized the XML file 'external_reviews.xml'. Now I able to run the 'Load XML INFILE'.

```
C:\Windows\system32\cmd.e. X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>cmd
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>copy "C:\Users\aylee\Downloads\external_reviews.xml" "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\"
1 file(s) copied.

C:\Users\aylee>dir "C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\"
Volume in drive C has no label.
Volume Serial Number is B0A1-EE81

Directory of C:\ProgramData\MySQL\MySQL Server 8.0\Uploads

2026-08-07  03:52 PM  <DIR>          .
2026-07-08  10:17 AM  <DIR>          ..
2026-08-07  12:05 PM                13,741 external_reviews.xml
               1 File(s)              13,741 bytes
               2 Dir(s)  360,051,855,360 bytes free

C:\Users\aylee>
```

026	
027	• SHOW GLOBAL VARIABLES LIKE 'local_infile';
028	

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Variable_name	Value
local_infile	ON

```
C:\Windows\system32\cmd.e. X + v
C:\Users\aylee>cmd
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aylee>"C:\Program Files\MySQL\MySQL Server 8.0"
"C:\Program Files\MySQL\MySQL Server 8.0" is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aylee>"C:\Program Files\MySQL\MySQL Server 8.0"
"C:\Program Files\MySQL\MySQL Server 8.0" is not recognized as an internal or external command,
operable program or batch file.

C:\Users\aylee>cd /d "C:\Program Files\MySQL\MySQL Server 8.0\bin"

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql --local-infile=1 -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 31
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```

027 • SHOW GLOBAL VARIABLES LIKE 'local_infile';
028 • SET GLOBAL local_infile = 0;
029
030 /*Part 2: Designing the Warehouse
031 Design a star schema for feedback analysis. You
032 set of dimensions. Draw it, a simple diagram is

```

Output

Action Output

#	Time	Action
947	15:44:57	SELECT COUNT(*) AS total_tickets, SUM(CASE WHEN customer_id IS NULL THEN
948	15:45:18	SELECT COUNT(*) AS total_tickets, SUM(CASE WHEN c.customerid IS NULL THEN
949	07:42:15	SHOW GLOBAL VARIABLES LIKE 'local_infile'
950	07:48:34	SET GLOBAL local_infile = 0

```

027 • SHOW GLOBAL VARIABLES LIKE 'local_infile';
028 • SET GLOBAL local_infile = 0;
029 • SHOW GLOBAL VARIABLES LIKE 'local_infile';

```

Result Grid

Filter Rows: Export: Wrap Cell Content:

Variable_name	Value
local_infile	OFF

And after all these processes I can load xml file and run.

```

298 • LOAD XML INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/external_reviews.xml'
299 INTO TABLE external_reviews_native
300 ROWS IDENTIFIED BY '<review>';
301
302 /*check the load xml file*/
303 • SELECT * FROM external_reviews_native LIMIT 10;
304
305 /*Adding missing columns*/

```

Result Grid

Filter Rows: Export: Wrap Cell Content: Fetch rows:

id	postedOn	displayName	email	country	item	score	body	helpfulVotes	sku	score
RM-7000	2024-11-11	jweber	jonas.weber@riverbox.com	CA	Cascade 1.7L Electric Kettle	3	Two week delay and nobody an...	8	HK-1041	10
RM-7001	2024-03-08	dwhitfield	dana.whitfield@corvid.io	MX	Auralis Studio Over-Ear Headp...	8	Assembly took ten minutes wit...	3	EL-2024	10
RM-7002	2024-08-14	cruiz	caleb.ruiz@brightpost.net	DE	Cascade 1.7L Electric Kettle	6	Happy with it. Minor niggles bu...	30	HK-1041	10
RM-7003	2024-02-15	tpetrov	teo.petrov@mailhaven.com	GB	Verdant Repair Night Cream	10	Nicely made. Weight and balan...	31	PC-3022	10
RM-7004	2024-08-11	emacchatti	emacchatti@mailhaven.com	US	Brinkle Beard & Hair Oil	10	My partner liked it enough to st...	30	PC-3047	10

5. **Clean with UPDATE statements**, each one separate and commented. At minimum, handle: inconsistent case and whitespace, a column containing two date formats, ratings outside the valid range, a rating scale that differs from the other feeds, and duplicate submissions.

```

382  /*5.Clean with UPDATE statements, each one separate and commented.
383  At minimum, handle:
384  /*inconsistent case and whitespace email web_feedback and external_reviews add 'unknown'*/
385  • SET SQL_SAFE_UPDATES = 0;
386  • UPDATE web_feedback_from_json
387  SET customer_email = 'unknown'
388  WHERE customer_email = '' OR customer_email IS NULL;
389
390  • UPDATE external_reviews_native
391  SET email = 'unknown'
392  WHERE email = '' OR email IS NULL;
393  • SET SQL_SAFE_UPDATES = 1;
394  /*Retrieve all changes*/
395  • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native; SELECT * FROM web_feedback_from_json;

```

submission_id	received_at	customer_name	customer_email	verified	product_sku	product_title	score	message
WF-5000	2024-10-29T21:48:00Z	Sofia Marchetti	sofia.marchetti@mailhaven.com	True	OG-5041	Inkwell A5 Hardcover Notebook	2	Manual is a poor translation and h
WF-5001	2024-12-23T16:23:00Z	Silas Lindgren	silas.lindgren@riverbox.com	True	OG-5012	Latitude Standing Desk Converter	2	Capacity is nowhere near what the
WF-5002	2024-04-01T12:00:00Z	Clara Duarte	clara.duarte@corvid.io	True	EL-2011	Auralis A2 Wireless Earbuds	3	Battery life is fine but the charging
WF-5003	2024-05-11T10:55:00Z	Clara Duarte	clara.duarte@corvid.io	True	OF-4056	Summit Foam Roller	5	Came a day earlier than the estima
WF-5004	2024-06-30T19:50:00Z	Devin Halloran	devin.halloran@brightpost.net	True	OG-5023	Latitude Ergonomic Mesh Chair	4	Good value at the sale price, I wou
WF-5005	2024-01-16T07:22:00Z	Hiro Tanaka	hiro.tanaka@mailhaven.com	True	OF-4031	Kestrel Resistance Band Set	2	Plastic feels cheap and the finish is
WF-5006	2024-12-03T16:05:00Z	Petra Kovac	petra.kovac@mailhaven.com	True	PC-3010	Verdant Daily Facial Cleanser	4	Runs large. Worth checking the me
WF-5007	2024-11-21T14:11:00Z	Joel Abbott	unknown	False	OF-4013	Trailhead 32L Daypack	5	Exactly what I needed. Nothing mc

```

90  • UPDATE external_reviews_native
91  SET email = 'unknown'
92  WHERE email = '' OR email IS NULL;
93  • SET SQL_SAFE_UPDATES = 1;
94  /*Retrieve all changes*/
95  • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native; SELECT * FROM web_feedback_from_j

```

id	postedOn	displayName	email	country	item	score	body
RM-7000	2024-11-11	jweber	jonas.weber@riverbox.com	CA	Cascade 1.7L Electric Kettle	3	Two week delay and r
RM-7001	2024-03-08	dwhitfield	dana.whitfield@corvid.io	MX	Auralis Studio Over-Ear Headp...	8	Assembly took ten mi
RM-7002	2024-08-14	cruiz	caleb.ruiz@brightpost.net	DE	Cascade 1.7L Electric Kettle	6	Happy with it. Minor r
RM-7003	2024-02-15	tpetrov	teo.petrov@mailhaven.com	GB	Verdant Repair Night Cream	10	Nicely made. Weight a
RM-7004	2024-08-11	smarchetti	sofia.marchetti@mailhaven.com	US	Bramble Beard & Skin Oil	10	My partner liked it enc
RM-7005	2024-10-18	iferreira	ines.ferreira@corvid.io	AU	Voltmark 20000mAh Power Bank	10	My partner liked it enc
RM-7006	2024-06-20	nadeyemi	unknown	GB	Verdant Repair Night Cream	10	Third one of these I h
RM-7007	2024-02-24	smarchetti	sofia.marchetti@mailhaven.com	GB	Trailhead Insulated Bottle 750ml	8	Good quality overall, i

```

396  /*set consistency respondent_name into customer_survey */
397  • SET SQL_SAFE_UPDATES = 0;
398  • UPDATE Customer_survey
399  SET respondent_name = CONCAT(
400  UPPER(SUBSTRING(SUBSTRING_INDEX(respondent_name, ' ', 1), 1, 1)),
401  LOWER(SUBSTRING(SUBSTRING_INDEX(respondent_name, ' ', 1), 2)),
402  ' ',
403  UPPER(SUBSTRING(SUBSTRING_INDEX(respondent_name, ' ', -1), 1, 1)),
404  LOWER(SUBSTRING(SUBSTRING_INDEX(respondent_name, ' ', -1), 2))
405  )
406  WHERE respondent_name LIKE '% %';
407

```

response_id	submitted_on	respondent_name	respondent_email	product_label	sku	rating_1_5	open_comment	channel
SV-2015	2024-01-19	Amara Okafor	amara.okafor@mailhaven.c...	Trailhead Insulated Bottle 750ml	OF-4025	4	Good quality overall, minor scu...	email survey
SV-2016	2024-06-14	Nia Adeyemi	nia.adeyemi@penpost.co	Cascade Pour-Over Coffee Set	HK-1052	2	Ordered on the 3rd, turned up ...	email survey
SV-2017	2024-01-09	Tomas Vesely	tomas.vesely@riverbox.com	Kestrel Resistance Band Set	OF-4031	3	Does the job but the finish is av...	email survey
SV-2018	2024-09-12	Sanne Vries	sanne.devries@brightpost.net	Meadowlark Ceramic Dinner Set...	HK-1090	5	Came a day earlier than the esti...	post-purchase
SV-2019	2024-12-01	Hiro Tanaka	hiro.tanaka@mailhaven.com	Kestrel Adjustable Dumbbell 24kg	OF-4044	5	Would buy again at this , Came a day earlier than the estimate w	
SV-2020	2024-07-17	Noor Rahimi	noor.rahimi@mailhaven.com	Voltmark 65W GaN Charger	EL-2036	1	Return took five weeks to proce...	post-purchase
SV-2021	2024-06-23	Teo Petrov	teo.petrov@mailhaven.com	Ridgeline Bamboo Cutting Board	HK-1078	2	Capacity is nowhere near what ...	post-purchase
SV-2022	2024-03-12	Emilia Reyes	emilia.reyes@mailhaven.c...	Verdant Daily Facial Cleanser	PC-3010	5	Cheapest face product I tried...	post-purchase

```

412  /*Check on rows counts*/
413 • SELECT COUNT(*) FROM Customer_survey;
414 • SELECT COUNT(*) FROM external_reviews_native;
415 • SELECT COUNT(*) FROM web_feedback_from_json;
416  /*Fix date formats a column containing two date formats*/
417  /*Retrieve all changes*/
418 • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native; SELECT
419  /*check any row where day first vs month */

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	COUNT(*)
▶	61

```

412  /*Check on rows counts*/
413 • SELECT COUNT(*) FROM Customer_survey;
414 • SELECT COUNT(*) FROM external_reviews_native;
415 • SELECT COUNT(*) FROM web_feedback_from_json;
416  /*Fix date formats a column containing two date formats*/
417  /*Retrieve all changes*/
418 • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native
419  /*check any row where day first vs month */

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	COUNT(*)
▶	33

```

412  /*Check on rows counts*/
413 • SELECT COUNT(*) FROM Customer_survey;
414 • SELECT COUNT(*) FROM external_reviews_native;
415 • SELECT COUNT(*) FROM web_feedback_from_json;
416  /*Fix date formats a column containing two date formats*/
417  /*Retrieve all changes*/
418 • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native; SELECT * FROM web_feedback_from_json;
419  /*check any row where day first vs month */

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	COUNT(*)
▶	38

```

417  /*Retrieve all changes*/
418 • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native; SELECT * FROM
419  /*check any row where day first vs month */
420 • SELECT submitted_on
421  FROM customer_survey
422  WHERE submitted_on LIKE '___/___/___'
423  ORDER BY submitted_on;
424
425  /*check the update*/

```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	response_id	submitted_on	respondent_name	respondent_email	product_label	sku
▶	SV-2000	2024-03-26	Tessa Vos	tessa.vos@penpost.co	Nimbus Indoor Air Monitor (EL...	EL-20
	SV-2001	2024-10-14	Nadia Hassan	nadia.hassan@brightpost.net	Voltmark 20000mAh Power Ba...	EL-20
	SV-2002	2024-08-21	Nadia Hassan	nadia.hassan@brightpost.net	Kestrel Resistance Band Set (OF...	OF-4C
	SV-2003	2024-09-18	Mira Kaur	mira.kaur@corvid.io	Auralis A2 Wireless Earbuds	EL-20
	SV-2004	2025-01-09	Malik Boateng	malik.boateng@penpost.co	Meadowlark Ceramic Dinner Set	HK-1C
	SV-2005	2025-03-06	Ravi Iyer	ravi.iver@mailhaven.com	Cascade 1.7l Electric Kettle	HK-1C

```

425  /*check the update*/
426 • SELECT submitted_on FROM customer_survey LIMIT 20;
427  /*Update the wrong dates formats*/
428 • SELECT * FROM customer_survey;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
submitted_on				
2024-03-26				
2024-10-14				
2024-08-21				
2024-09-18				
2025-01-09				
2025-03-06				
2025-02-12				
2025-05-22				


```
140  /*ratings outside the valid range*/
141  /*check first the values*/
142  • SELECT rating_1_5, COUNT(*) AS occurrences
143  FROM customer_survey
144  GROUP BY rating_1_5
145  ORDER BY rating_1_5;
146  /*isolate the problem*/
147  • SELECT *
148  FROM customer_survey
149  WHERE rating_1_5 NOT BETWEEN 1 AND 5
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [1A](#)

	rating_1_5	occurrences
▶	0	1
	1	5
	2	13
	3	12
	4	10
	5	21
	7	1

```
146  /*isolate the problem*/
147  • SELECT *
148  FROM customer_survey
149  WHERE rating_1_5 NOT BETWEEN 1 AND 5
150  OR rating_1_5 IS NULL;
151  /*Set out of range values to null*/
152  • SET SQL_SAFE_UPDATES =0;
153  • UPDATE customer_survey
154  SET rating_1_5 = NULL
155  WHERE rating_1_5 NOT BETWEEN 1 AND 5;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [1A](#)

	response_id	submitted_on	respondent_name	respondent_email	product_label	sku	rating_1_5	open_comment	channel
▶	SV-2049	2024-12-17	NIA ADEYEMI	nia.adeyemi@penpost.co	Kestrel Adjustable Dumbbell 24...	OF-4044	7	Cheaper than the brand-name e...	post-purchase
	SV-2057	2025-04-24	GRETA LINDQVIST	greta.lindqvist@brightpost.net	Voltmark 20000mAh Power Ba...	EL-2048	0	Two of the pieces arrived chipp...	post-purchase

```

450         OR rating_1_5 IS NULL;
451     /*Set out of range values to null*/
452 • SET SQL_SAFE_UPDATES =0;
453 • UPDATE customer_survey
454     SET rating_1_5 = NULL
455     WHERE rating_1_5 NOT BETWEEN 1 AND 5;
456 • SET SQL_SAFE_UPDATES =1;
457     /*Call/check the result*/
458 • SELECT rating_1_5, COUNT(*) AS occurrences
459     FROM customer_survey
460     WHERE rating_1_5 IS NOT NULL
461     GROUP BY rating_1_5

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	rating_1_5	occurrences		
▶	1	5		
	2	13		
	3	12		
	4	10		
	5	21		

```

451     /*Set out of range values to null*/
452 • SET SQL_SAFE_UPDATES =0;
453 • UPDATE customer_survey
454     SET rating_1_5 = NULL
455     WHERE rating_1_5 NOT BETWEEN 1 AND 5;
456 • SET SQL_SAFE_UPDATES =1;
457     /*Call/check the result*/
458 • SELECT rating_1_5, COUNT(*) AS occurrences
459     FROM customer_survey
460     WHERE rating_1_5 IS NOT NULL
461     GROUP BY rating_1_5
462     ORDER BY rating_1_5;
463     /*check the nulls rows*/
464 • SELECT * FROM customer_survey WHERE rating_1_5 IS NULL;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	rating_1_5	occurrences		
▶	1	5		
	2	13		
	3	12		
	4	10		
	5	21		

```

463  /*check the nulls rows*/
464 • SELECT * FROM customer_survey WHERE rating_1_5 IS NULL;

```

response_id	submitted_on	respondent_name	respondent_email	product_label	sku	rating_1_5	open_comment	channel
SV-2049	2024-12-17	NIA ADEYEMI	nia.adeyemi@penpost.co	Kestrel Adjustable Dumbbell 24...	OF-4044		Cheaper than the brand-name e...	post-purchase
SV-2057	2025-04-24	GRETA LINDQVIST	greta.lindqvist@brightpost.net	Voltmark 20000mAh Power Ba...	EL-2048		Two of the pieces arrived chipp...	post-purchase

```

465
466  /*a rating scale that differs from the other feeds*/
467  /*step 1:Check the survey's declared scale*/
468 • SELECT rating_1_5 FROM customer_survey ORDER BY rating_1_5;
469  /*Delete the null*/
470 • SET SQL_SAFE_UPDATES = 0;
471 • DELETE FROM customer_survey
472   WHERE rating_1_5 IS NULL;
473 • SET SQL_SAFE_UPDATES = 1;

```

rating_1_5
1
1
1
1
1
2
2

customer_survey 411 x

```

466  /*a rating scale that differs from the other feeds*/
467  /*step 1:Check the survey's declared scale*/
468 • SELECT rating_1_5 FROM customer_survey ORDER BY rating_1_5;
469  /*Delete the null*/
470 • SET SQL_SAFE_UPDATES = 0;
471 • DELETE FROM customer_survey
472   WHERE rating_1_5 IS NULL;
473 • SET SQL_SAFE_UPDATES = 1;
474  /*check the rating*/
475 • SELECT DISTINCT rating_1_5 FROM customer_survey ORDER BY rating_1_5;
476

```

rating_1_5
1
2
3
4
5

```

477  /*step 2:Check the XML source's actual scale attribute*/
478 • SELECT DISTINCT score_scale FROM external_reviews_native;
479  /*Apply normalization add new column score_normalized*/
480 • ALTER TABLE external_reviews_native ADD COLUMN score_normalized DECIMAL
481
482 • UPDATE external_reviews_native

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
score_scale			
10			

```

477  /*step 2:Check the XML source's actual scale attribute*/
478 • SELECT DISTINCT score_scale FROM external_reviews_native;
479  /*Apply normalization add new column score_normalized*/
480 • ALTER TABLE external_reviews_native ADD COLUMN score_normalized DECIMAL(3,1);
481
482 • UPDATE external_reviews_native
483   SET score_normalized = (score / score_scale) * 5;
484  /*Verify the output*/
485 • SELECT score, score_scale, score_normalized FROM external_reviews_native LIMIT 10;
486  /*Duplicate submissions.The duplicates do not share a record ID, so you will need to define a b

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
score	score_scale	score_normalized		
3	10	1.5		
8	10	4.0		
6	10	3.0		
10	10	5.0		
10	10	5.0		
10	10	5.0		
10	10	5.0		
8	10	4.0		
10	10	5.0		

external_reviews_native 348 x

```

486  /*Duplicate submissions.The duplicates do not share a record ID, so you will need to define a business key to find them.*/
487  /*Find duplicated*/
488 • SELECT respondent_email, product_label, rating_1_5, open_comment, COUNT(*) AS occurrences
489   FROM customer_survey
490  GROUP BY respondent_email, product_label, rating_1_5, open_comment
491  HAVING COUNT(*) > 1;
492  /*confirm if those records are genuine duplicate submission*/
493 • SELECT c.*
494   FROM customer_survey c
495  JOIN (
496   SELECT respondent_email, product_label, rating_1_5, open_comment

```

Result Grid		Filter Rows:	Export:	Wrap Cell Contents:	
	respondent_email	product_label	rating_1_5	open_comment	occurrences
▶	lena.novak@mailhaven.com	Kestrel Adjustable Dumbbell 24kg	3	Fine for occasional use, I would...	2
	noor.rahimi@mailhaven.com	Ridgeline Bamboo Cutting Boar...	2	Held up fine for the first two m...	2
	jonas.weber@riverbox.com	Auralis A2 Wireless Earbuds (E...	2	Support was slow but did event...	2

```

492  /*confirm if those records are genuine duplicate submission*/
493  • SELECT c.*
494  FROM customer_survey c
495  JOIN (
496      SELECT respondent_email, product_label, rating_1_5, open_comment
497      FROM customer_survey
498      GROUP BY respondent_email, product_label, rating_1_5, open_comment
499      HAVING COUNT(*) > 1
500  ) dup
501  ON c.respondent_email = dup.respondent_email
502  AND c.product_label = dup.product_label
503  AND c.rating_1_5 = dup.rating_1_5
504  AND c.open_comment = dup.open_comment
505  ORDER BY c.respondent_email;

```

	response_id	submitted_on	respondent_name	respondent_email	product_label	sku	rating_1_5	open_comment	channel
▶	SV-92041	2024-11-18	Jonas Weber	jonas.weber@riverbox.com	Auralis A2 Wireless Earbuds (E...	EL-2011	2	Support was slow but did event...	post-purchase
	SV-2041	2024-11-18	Jonas Weber	jonas.weber@riverbox.com	Auralis A2 Wireless Earbuds (E...	EL-2011	2	Support was slow but did event...	post-purchase
	SV-92011	2025-01-06	Lena Novak	lena.novak@mailhaven.com	Kestrel Adjustable Dumbbell 24kg	OF-4044	3	Fine for occasional use, I would...	Email Survey
	SV-2011	2025-01-06	Lena Novak	lena.novak@mailhaven.com	Kestrel Adjustable Dumbbell 24kg	OF-4044	3	Fine for occasional use, I would...	email survey
	SV-92038	2024-04-11	Noor Rahimi	noor.rahimi@mailhaven.com	Ridgeline Bamboo Cutting Boar...	HK-1078	2	Held up fine for the first two m...	email survey
	SV-2038	2024-04-11	Noor Rahimi	noor.rahimi@mailhaven.com	Ridgeline Bamboo Cutting Boar...	HK-1078	2	Held up fine for the first two m...	email survey

```

105  ORDER BY c.respondent_email;
106  /*Delete the duplicates and keep the earliest one*/
107  • SET SQL_SAFE_UPDATES =0;
108  • DELETE c1 FROM customer_survey c1
109  JOIN customer_survey c2
110  ON c1.respondent_email = c2.respondent_email
111  AND c1.product_label = c2.product_label
112  AND c1.rating_1_5 = c2.rating_1_5
113  AND c1.open_comment = c2.open_comment
114  AND c1.response_id > c2.response_id;
115  • SET SQL_SAFE_UPDATES =1;
116  /*Verify the clean*/
117  • SELECT respondent_email, product_label, rating_1_5, open_comment, COUNT(*) AS occurrences
118  FROM customer_survey
119  GROUP BY respondent_email, product_label, rating_1_5, open_comment
120  HAVING COUNT(*) > 1;
121  /*6. Log anything you reject or null out, with a reason, in a reject table. Deleting silently ma

```

	respondent_email	product_label	rating_1_5	open_comment	occurrences
--	------------------	---------------	------------	--------------	-------------

The duplicates do *not* share a record ID, so you will need to define a business key to find them.

6. Log anything you reject or null out, with a reason, in a reject table. Deleting silently makes your row counts impossible to reconcile.

```

521  /*6. Log anything you reject or null out, with a reason, in a reject table. Deleting silently makes your row
522  • CREATE TABLE dim_customer (
523      customer_id INT AUTO_INCREMENT PRIMARY KEY,
524      email VARCHAR(255) NOT NULL UNIQUE,
525      customer_name VARCHAR(255)
526  );
527  • CREATE TABLE fact_review (
528      review_id INT AUTO_INCREMENT PRIMARY KEY,
529      source_system VARCHAR(30) NOT NULL,
530      source_id VARCHAR(50) NOT NULL,
531      customer_id INT NULL,
532      product_sku VARCHAR(20),
533      product_title VARCHAR(255),
534      score_raw DECIMAL(4,1),
535      score_normalized DECIMAL(3,1),
536      review_date DATETIME,
537      review_text TEXT,
538      channel VARCHAR(50),
539      country VARCHAR(5),

```

```

i46  /*Create Reject_log*/
i47  • CREATE TABLE reject_log (
i48      reject_id INT AUTO_INCREMENT PRIMARY KEY,
i49      source_system VARCHAR(30) NOT NULL,
i50      source_id VARCHAR(50) NOT NULL,
i51      column_name VARCHAR(50) NOT NULL,
i52      original_value VARCHAR(255),
i53      reject_reason VARCHAR(255) NOT NULL,
i54      logged_at DATETIME DEFAULT CURRENT_TIMESTAMP
i55  );

```

```

557  /*Retrieve all changes*/
558  • SELECT * FROM Customer_survey; SELECT * FROM external_reviews_native; SELECT * FROM web_feedback_from_json;
559  /*Create the Fact_reviews combine all tables in ones*/
560  -- Customer_survey = fact_review
561  • INSERT INTO fact_review
562      (source_system, source_id, customer_id, product_sku, product_title,
563      score_raw, score_normalized, review_date, review_text, channel)
564  SELECT
565      'customer_survey', s.response_id, c.customer_id, s.sku, s.product_label,
566      s.rating 1 5, s.rating 1 5, s.submitted on, s.open comment, s.channel

```

```

569 ON DUPLICATE KEY UPDATE customer_id = c.customer_id;
570 -- external_reviews_native = fact_review
571 • INSERT INTO fact_review
572     (source_system, source_id, customer_id, product_sku, product_title,
573      score_raw, score_normalized, review_date, review_text, channel, country, helpful_votes)
574 SELECT
575     'external_reviews', e.id, c.customer_id, e.sku, e.item,
576     e.score, e.score_normalized, e.postedOn, e.body, 'external', e.country, e.helpfulVotes
577 FROM external_reviews_native e
578 LEFT JOIN dim_customer c ON c.email = LOWER(TRIM(e.email))
579 ON DUPLICATE KEY UPDATE customer_id = c.customer_id;
580 -- web_feedback_from_json = fact_review
581 • INSERT INTO fact_review

```

```

581 • INSERT INTO fact_review
582     (source_system, source_id, customer_id, product_sku, product_title,
583      score_raw, review_date, review_text, verified)
584 SELECT
585     'web_feedback', w.submission_id, c.customer_id, w.product_sku, w.product_title,
586     w.score,
587     STR_TO_DATE(REPLACE(REPLACE(w.received_at, 'T', ' '), 'Z', ''), '%Y-%m-%d %H:%i:%s'),
588     w.message,
589     CASE WHEN w.verified = 'True' THEN 1 ELSE 0 END
590 FROM web_feedback_from_json w
591 LEFT JOIN dim_customer c ON c.email = LOWER(TRIM(w.customer_email))
592 ON DUPLICATE KEY UPDATE customer_id = c.customer_id;
593

```

```

---
594 /* Verify: row counts per source*/
595 • SELECT source_system, COUNT(*) AS promoted_rows
596 FROM fact_review
597 GROUP BY source_system;
598
599 /*Check the tables */

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	source_system	promoted_rows
▶	customer_survey	61
	external_reviews	33
	web_feedback	38

```

610 /*check the new table*/
611 • SELECT * FROM vw_all_reviews_combined;
612 • SELECT COUNT(*) FROM vw_all_reviews_combined;
613 /*confirm all three sources are present==*/

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	COUNT(*)
▶	132

- Promote staging into your final tables. Reviews with no matching customer must still load.

```
609  /*check the new table*/
610 • SELECT * FROM vw_all_reviews_combined;
611 • SELECT COUNT(*) FROM vw_all_reviews_combined;
612  /*confirm all three sources are present*/
613 • SELECT source_system, COUNT(*) AS row_count
614 FROM vw_all_reviews_combined
```

source_id	source_system
SV-2000	customer_survey
SV-2001	customer_survey
SV-2002	customer_survey
SV-2003	customer_survey
SV-2004	customer_survey
SV-2005	customer_survey
SV-2006	customer_survey
SV-2007	customer_survey
SV-2008	customer_survey

```
610  /*check the new table*/
611 • SELECT * FROM vw_all_reviews_combined;
612 • SELECT COUNT(*) FROM vw_all_reviews_combined;
613  /*confirm all three sources are present===*/
```

COUNT(*)
132

```
613  /*confirm all three sources are present===*/
614 • SELECT source_system, COUNT(*) AS row_count
615 FROM vw_all_reviews_combined
616 GROUP BY source_system;
617
618  /*Part 3: Queries, Optimization and Testing*/
619 • SELECT * FROM Customers; SELECT * FROM reviews; SELECT * FROM products;
```

source_system	row_count
customer_survey	61
external_reviews	33
web_feedback	38

Part 3: Queries, Optimization and Testing

1. **Create a combined view** that joins customer, product, and review data. Use **LEFT JOIN** where a relationship may be missing, so no review drops out because it has no customer. Check it: the view must return the same row count as the review table.

```
86  /*Create the Fact_reviews combine all tables in ones*/
87  -- Customer_survey = fact_review
88  • INSERT INTO fact_review
89    (source_system, source_id, customer_id, product_sku, product_title,
90     score_raw, score_normalized, review_date, review_text, channel)
91  SELECT
92    'customer_survey', s.response_id, c.customer_id, s.sku, s.product_label,
93    s.rating_1_5, s.rating_1_5, s.submitted_on, s.open_comment, s.channel
94  FROM Customer_survey s
95  LEFT JOIN dim_customer c ON c.email = LOWER(TRIM(s.respondent_email))
96  ON DUPLICATE KEY UPDATE customer_id = c.customer_id;
97  -- external_reviews_native = fact_review
98  INSERT INTO fact_review
99    (source_system, source_id, customer_id, product_sku, product_title,
100   score_raw, score_normalized, review_date, review_text, channel, country, helpful_votes)
101  SELECT
102    'external_reviews', e.id, c.customer_id, e.sku, e.item,
103    e.score, e.score_normalized, e.postedOn, e.body, 'external', e.country, e.helpfulVotes
104  FROM external_reviews_native e
105  LEFT JOIN dim_customer c ON c.email = LOWER(TRIM(e.email))
106  ON DUPLICATE KEY UPDATE customer_id = c.customer_id;
107  -- web_feedback_from_json = fact_review
108  • INSERT INTO fact_review
109    (source_system, source_id, customer_id, product_sku, product_title,
110     score_raw, review_date, review_text, verified)
111  SELECT
112    'web_feedback', w.submission_id, c.customer_id, w.product_sku, w.product_title,
113    w.score,
114    STR_TO_DATE(REPLACE(REPLACE(w.received_at, 'T', ' '), 'Z', ''), '%Y-%m-%d %H:%i:%s'),
115    w.message,
116    CASE WHEN w.verified = 'True' THEN 1 ELSE 0 END
117  FROM web_feedback_from_json w
118  LEFT JOIN dim_customer c ON c.email = LOWER(TRIM(w.customer_email))
119  ON DUPLICATE KEY UPDATE customer_id = c.customer_id;
120  /* Verify: row counts per source*/
121  • SELECT source_system, COUNT(*) AS promoted_rows
122  FROM fact_review
123  GROUP BY source_system;
124  ...
```

2. **Write the analysis queries** using **WHERE**, **GROUP BY**, **HAVING** and **ORDER BY**:
 - Top products by rating *and* sales. Ranking on raw **AVG(rating)** puts a two-review product on top; address this with a review threshold or a weighted score.

```

530  /*Part 3: Queries, Optimization and Testing*/
531 • SELECT * FROM Customers; SELECT * FROM reviews; SELECT * FROM products;
532  /*Top products by rating and sales. Ranking on raw AVG(rating) puts a two-review product
533 • SELECT p.productid, p.name, p.category, COUNT(r.reviewid) AS review_count,
534         AVG(r.rating) AS avg_rating, SUM(p.price) AS sales_proxy
535  FROM Products p
536  JOIN Reviews r ON r.productid = p.productid
537  GROUP BY p.productid, p.name, p.category
538  HAVING COUNT(r.reviewid) >= 1
539  ORDER BY avg_rating DESC, review_count DESC
540  LIMIT 20;
541

```

productid	name	category	review_count	avg_rating	sales_proxy
1	Wireless Mouse	Electronics	1	5.0000	24.99
4	Ergonomic Office Chair	Furniture	1	5.0000	289.99
7	27-inch Monitor	Electronics	1	5.0000	249.00
9	USB-C Docking Station	Electronics	1	5.0000	129.99
3	Noise-Cancelling Headphones	Electronics	2	4.5000	399.98
2	Standing Desk	Furniture	1	4.0000	349.00
5	Mechanical Keyboard	Electronics	1	4.0000	89.99
8	Laptop Stand	Furniture	1	4.0000	45.99
6	LED Desk Lamp	Furniture	1	3.0000	34.50

- Recurring issues, grouped into categories. Store the keyword mapping as data in a table, not as a hard-coded CASE chain.

```

542  /*Recurring issues, grouped into categories. Store the keyword mapping as data in a table, not as a hard-coded CASE chain.*/
543  /*Create a keywords mapping table*/
544 • CREATE TABLE issue_keyword_map (
545      keyword_id INT AUTO_INCREMENT PRIMARY KEY,
546      keyword VARCHAR(100) NOT NULL,
547      category VARCHAR(100) NOT NULL
548  );
549  /*Populate the new table with keywords and category*/
550 • TRUNCATE TABLE issue_keyword_map;
551
552 • INSERT INTO issue_keyword_map (keyword, category) VALUES
553  ('responsive', 'Performance - Positive'),

```

```

549  /*Populate the new table with keywords and category*/
550 • TRUNCATE TABLE issue_keyword_map;
551
552 • INSERT INTO issue_keyword_map (keyword, category) VALUES
553   ('responsive', 'Performance - Positive'),
554   ('effective', 'Performance - Positive'),
555   ('sturdy', 'Build Quality - Positive'),
556   ('flimsy', 'Build Quality Issue'),
557   ('assembly', 'Assembly/Setup Issue'),
558   ('sound quality', 'Audio Quality - Positive'),
559   ('tactile', 'Feel/Tactile - Positive'),
560   ('loud', 'Noise Issue'),
561   ('brightness', 'Display Issue'),
562   ('crisp', 'Display Quality - Positive'),

```

```

/*Match reviews to categories and comments reviews*/
• SELECT r.reviewid, r.comments, k.category
  FROM Reviews r
 LEFT JOIN issue_keyword_map k
    ON r.comments LIKE CONCAT('%', k.keyword, '%')
 ORDER BY r.reviewid;
/*Retrieve count each category and numbers positive, issues and more*/
• SELECT k.category, COUNT(DISTINCT r.reviewid) AS mention_count

```

reviewid	comments	category
	Works great, very responsive.	Performance - Positive
	Sturdy but assembly took a while.	Assembly/Setup Issue
	Sturdy but assembly took a while.	Build Quality - Positive
	Excellent sound quality.	Audio Quality - Positive
	Good value for the price.	Value/Price - Positive
	Good value for the price.	Value/Price - Positive
	Very comfortable for long work hours.	Comfort - Positive
	Great tactile feedback, a bit loud.	Noise Issue
	Great tactile feedback, a bit loud.	Feel/Tactile - Positive

```

666  /*Retrieve count each category and numbers positive, issues and more*/
667  • SELECT k.category, COUNT(DISTINCT r.reviewid) AS mention_count
668  FROM Reviews r
669  JOIN issue_keyword_map k
670      ON r.comments LIKE CONCAT('%', k.keyword, '%')
671  GROUP BY k.category
672  ORDER BY mention_count DESC;
673  /*Rating trend by month, positive/negative split, keyword frequency, and customer ei

```

Result Grid	
Filter Rows:	Export: Wrap Cell Content:
category	mention_count
► Performance - Positive	3
Value/Price - Positive	2
Assembly/Setup Issue	1
Audio Quality - Positive	1
Build Quality - Positive	1
Build Quality Issue	1
Comfort - Positive	1
Display Issue	1

```

577  /*Retrieve count each category and numbers positive, issues and more*/
578  • SELECT k.category, COUNT(DISTINCT r.reviewid) AS mention_count
579  FROM Reviews r
580  JOIN issue_keyword_map k
581      ON r.comments LIKE CONCAT('%', k.keyword, '%')
582  GROUP BY k.category
583  ORDER BY mention_count DESC;
584  /*Rating trend by month, positive/negative split, keyword frequency, and custom
585  /*Rating trend by month*/
586  • SELECT DATE_FORMAT(Full_Date, '%Y-%m') AS review_month,

```

Result Grid	
Filter Rows:	Export: Wrap Cell Content:
category	mention_count
► Performance - Positive	3
Value/Price - Positive	2
Assembly/Setup Issue	1
Audio Quality - Positive	1
Build Quality - Positive	1
Build Quality Issue	1
Comfort - Positive	1
Display Issue	1
Display Quality - Positive	1

Result 256 x

- Rating trend by month, positive/negative split, keyword frequency, and customer engagement by region.

```

672 ORDER BY mention_count DESC;
673 /*Rating trend by month, positive/negative split, keyword frequency, and customer engagement by region.*/
674 /*Rating trend by month*/
675 • SELECT DATE_FORMAT(Full_Date, '%Y-%m') AS review_month,
676         COUNT(*) AS review_count,
677         ROUND(AVG(rating), 2) AS avg_rating
678 FROM Reviews
679 GROUP BY DATE_FORMAT(Full_Date, '%Y-%m')
680 ORDER BY review_month;
681 /*positive/negative split*/
682 • SELECT

```

review_month	review_count	avg_rating
2026-05	3	4.67
2026-06	4	4.25
2026-07	3	4.33

```

681 /*positive/negative split*/
682 • SELECT
683     CASE
684         WHEN rating >= 4 THEN 'Positive'
685         WHEN rating = 3 THEN 'Neutral'
686         ELSE 'Negative'
687     END AS sentiment,
688     COUNT(*) AS review_count,
689     ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Reviews), 1) AS pct_of_total
690 FROM Reviews
691 GROUP BY sentiment

```

sentiment	review_count	pct_of_total
Positive	9	90.0
Neutral	1	10.0

```

693  /*keyword frequency,*/
694  • SELECT k.category, k.keyword, COUNT(*) AS mention_count
695  FROM Reviews r
696  JOIN issue_keyword_map k
697      ON r.comments LIKE CONCAT('%', k.keyword, '%')
698  GROUP BY k.category, k.keyword
699  ORDER BY mention_count DESC;
700  /*Customer engagement by region*/
701  • SELECT c.region,
702          COUNT(r.reviewid) AS total_reviews,
703          COUNT(DISTINCT r.customerid) AS distinct_reviewers,

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	category	keyword	mention_count
▶	Value/Price - Positive	price	2
	Performance - Positive	flawlessly	2
	Performance - Positive	responsive	1
	Performance - Positive	effective	1
	Build Quality - Positive	sturdy	1
	Build Quality Issue	flimsy	1
	Assembly/Setup Issue	assembly	1
	Audio Quality - Positive	sound quality	1
	Feel/Tactile - Positive	tactile	1

```

3  /*Customer engagement by region*/
1  • SELECT c.region,
2      COUNT(r.reviewid) AS total_reviews,
3      COUNT(DISTINCT r.customerid) AS distinct_reviewers,
4      ROUND(AVG(r.rating), 2) AS avg_rating
5  FROM Customers c
5  LEFT JOIN Reviews r ON r.customerid = c.customerid
7  GROUP BY c.region
3  ORDER BY total_reviews DESC;
3  /*A comparison of the three sources. They will not agree – quantify by how much.*/

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

region	total_reviews	distinct_reviewers	avg_rating
Manitoba	3	2	4.00
British Columbia	2	2	5.00
Ontario	2	2	4.00
Alberta	1	1	5.00
Nova Scotia	1	1	5.00
Quebec	1	1	4.00
Saskatchewan	0	0	NULL

- A comparison of the three sources. They will not agree — quantify by how much.

```

620  /*A comparison of the three sources. They will not agree – quantify by how much.*/
621 • SELECT
622     sku,
623     COALESCE(CAST(survey_avg AS CHAR), 'No Data') AS survey_avg,
624     COALESCE(CAST(external_avg AS CHAR), 'No Data') AS external_avg,
625     COALESCE(CAST(web_avg AS CHAR), 'No Data') AS web_avg
626 FROM (
627     SELECT
628         sku,
629         ROUND(AVG(CASE WHEN source = 'survey' THEN score_0to5 END), 2) AS survey_avg,
630         ROUND(AVG(CASE WHEN source = 'external' THEN score_0to5 END), 2) AS external_avg,

```

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: |

	sku	survey_avg	external_avg	web_avg
▶	EL-2011	2	No Data	1.17
	EL-2024	4	4	1.5
	EL-2036	2.5	4	1
	EL-2048	4	4.25	1.83
	EL-2055	3	3	No Data
	EL-2067	4.67	2	2
	HK-1041	2	1.83	1.25
	HK-1063	5	3.25	2.17
	HK-1078	2.67	3.33	2.25

Result 261 x

Watch for this: reviews and sales sit at different grains. Joining both to the product table at once multiplies revenue by the review count.

3. **Create two views:** top-rated products, and flagged reviews. Define "flagged" yourself and justify the threshold.

Definition f “Flagged” a review is flagged if either of two conditions are true: example Low rating (1 or 2) and Negative keyword example(build quality issue, assembly/issue, etc).

```

664  /*3. Create two views: top-rated products, and flagged reviews. Define "flagged" yourself and justify the threshold.*/
665  /* top-rated products*/
666 • DROP VIEW IF EXISTS vw_top_rated_products;
667 • CREATE VIEW vw_top_rated_products AS
668     SELECT
669         p.productid, p.name, p.category, COUNT(r.reviewid) AS review_count,
670         ROUND(AVG(r.rating), 2) AS avg_rating
671     FROM Products p
672     JOIN Reviews r ON r.productid = p.productid
673     GROUP BY p.productid, p.name, p.category
674     HAVING COUNT(r.reviewid) >= 1
675     ORDER BY avg_rating DESC, review_count DESC;
676  /*flagged reviews*/
677 • CREATE VIEW vw_flagged_reviews AS
678     SELECT productid, name, category, review_count, avg_rating

```

```

663  /*flagged reviews*/
664 • CREATE VIEW vw_flagged_reviews AS
665      SELECT DISTINCT
666          r.reviewid,
667          r.customerid,
668          r.productid,
669          r.rating,
670          r.comments,
671          r.Full_Date,
672          CASE
673              WHEN r.rating <= 2 THEN 'Low Rating'
674              ELSE 'Negative Keyword'
675          END AS flag_reason

```

```

579      issue_keyword_map k ON r.comments LIKE CONCAT('%', k.keyword, '%')
580      AND k.category LIKE '%Issue%'
581  WHERE
582      r.rating <= 2 OR k.category IS NOT NULL;
583  /*Call both views */
584 • SELECT * FROM vw_top_rated_products; SELECT * FROM vw_flagged_reviews;

```

reviewid	customerid	productid	rating	comments	Full_Date	flag_reason
2	2	2	4	Sturdy but assembly took a while.	2026-06-01	Negative Keyword
6	5	5	4	Great tactile feedback, a bit loud.	2026-05-25	Negative Keyword
7	6	6	3	Decent brightness but flimsy base.	2026-06-10	Negative Keyword

```

681  /*Call both views */
682 • SELECT * FROM vw_top_rated_products; SELECT * FROM vw_flagged_reviews;

```

	productid	name	category	review_count	avg_rating
▶	1	Wireless Mouse	Electronics	1	5.00
	4	Ergonomic Office Chair	Furniture	1	5.00
	7	27-inch Monitor	Electronics	1	5.00
	9	USB-C Docking Station	Electronics	1	5.00
	3	Noise-Cancelling Headphones	Electronics	2	4.50
	2	Standing Desk	Furniture	1	4.00
	5	Mechanical Keyboard	Electronics	1	4.00
	8	Laptop Stand	Furniture	1	4.00
	6	LED Desk Lamp	Furniture	1	3.00

vw_top_rated_products 265 × vw_flagged_reviews 266

Output

4. **Add indexes** only where a query you wrote needs one, and name the query each serves. Create them *after* the bulk load and explain why the order matters. Run **EXPLAIN** before and after, and paste both into your report.


```

683 /*4.Add indexes only where a query you wrote needs one, and name the query each serves.
684 Create them after the bulk load and explain why the order matters. Run EXPLAIN before and after, and paste both into your report.*/
685 /*Explanation before queries*/
686 • EXPLAIN SELECT p.productid, p.name, p.category, COUNT(r.reviewid) AS review_count, AVG(r.rating) AS avg_rating
687 FROM Products p
688 JOIN Reviews r ON r.productid = p.productid
689 GROUP BY p.productid, p.name, p.category;
690
691 • EXPLAIN SELECT * FROM Reviews WHERE rating <= 2;
692
693 • EXPLAIN SELECT c.region, COUNT(r.reviewid), AVG(r.rating)
694 FROM Customers c
695 LEFT JOIN Reviews r ON r.customerid = c.customerid

```

```

774 /*Explanation before queries*/
775 • EXPLAIN SELECT p.productid, p.name, p.category, COUNT(r.reviewid) AS review_count, AVG(r.rating) AS avg_rating
776 FROM Products p
777 JOIN Reviews r ON r.productid = p.productid
778 GROUP BY p.productid, p.name, p.category;
779
780 • EXPLAIN SELECT * FROM Reviews WHERE rating <= 2;
781
782 • EXPLAIN SELECT c.region, COUNT(r.reviewid), AVG(r.rating)
783 FROM Customers c
784 LEFT JOIN Reviews r ON r.customerid = c.customerid

```

Result Grid													Filter Rows:	Exports:	Wrap Cell Content:
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra			
▶	1	SIMPLE	r	HULL	ALL	idx_reviews_productid	HULL	HULL	HULL	10	100.00	Using temporary			
	1	SIMPLE	p	HULL	eq_ref	PRIMARY	PRIMARY	4	e_commercecompanydb.r.prod...	1	100.00	HULL			

```

779
780 • EXPLAIN SELECT * FROM Reviews WHERE rating <= 2;
781
782 • EXPLAIN SELECT c.region, COUNT(r.reviewid), AVG(r.rating)
783 FROM Customers c
784 LEFT JOIN Reviews r ON r.customerid = c.customerid

```

Result Grid												Filter Rows:	Export:	Wrap Cell Content:
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra		
▶	1	SIMPLE	Reviews	HULL	range	idx_reviews_rating	idx_reviews_rating	4	HULL	1	100.00	Using index condition		

```

782 • EXPLAIN SELECT c.region, COUNT(r.reviewid), AVG(r.rating)
783 FROM Customers c
784 LEFT JOIN Reviews r ON r.customerid = c.customerid
785 GROUP BY c.region;
786
787 • EXPLAIN SELECT DATE_FORMAT(Full_Date, '%Y-%m') AS review_month, COUNT(*), AVG(rating)

```

Result Grid													Filter Rows:		Export:		Wrap Cell Content:			
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra								
▶	1	SIMPLE	c	HULL	index	idx_customers_region	idx_customers_region	402		10	100.00	Using index								
	1	SIMPLE	r	HULL	ref	uq_customer_product_review	uq_customer_product_review	4	e_commercecompanydb.c.cust...	1	100.00	HULL								

```

813 • EXPLAIN SELECT DATE_FORMAT(Full_Date, '%Y-%m') AS review_month, COUNT(*), AVG(rating)
814 FROM Reviews
815 GROUP BY DATE_FORMAT(Full_Date, '%Y-%m');
816
817 /*5. Write validation queries that return a clear pass or fail: rows staged equals rows loaded plus rejected;
818 no record loaded twice; all ratings in range; no orphaned foreign keys; all dates plausible;
819 the combined view row count matches the review table; aggregate totals reconcile to the underlying rows.*/
820 /*rows staged equals rows loaded plus rejected*/

```

Result Grid													Filter Rows:	Export:	Wrap Cell Content:
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra			
▶	1	SIMPLE	Reviews	HULL	ALL	idx_reviews_date,idx_reviews_f...	HULL	HULL	HULL	10	100.00	Using temporary			

```

786
787 • EXPLAIN SELECT DATE_FORMAT(Full_Date, '%Y-%m') AS review_month, COUNT(*), AVG(rating)
788 FROM Reviews
789 GROUP BY DATE_FORMAT(Full_Date, '%Y-%m');
790 /*Create indexes*/
791 /*JOIN in vw_top_rated_products, and any Reviews=Products join*/
792 • CREATE INDEX idx_reviews_productid ON Reviews(productid);
793 /*WHERE rating <= 2 filter in vw_flagged_reviews*/
794 • CREATE INDEX idx_reviews_rating ON Reviews(rating);
795 /*GROUP BY region in customer engagement query*/
796 • CREATE INDEX idx_customers_region ON Customers(region);

```

Result Grid												Filter Rows:		Export:		Wrap Cell Content:	
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra					
▶	1	SIMPLE	Reviews	HULL	ALL	idx_reviews_date,idx_reviews_f...	HULL	HULL	HULL	10	100.00	Using temporary					

```

701 /*Create indexes*/
702 /*JOIN in vw_top_rated_products, and any Reviews=Products join*/
703 • CREATE INDEX idx_reviews_productid ON Reviews(productid);
704 /*WHERE rating <= 2 filter in vw_flagged_reviews*/
705 • CREATE INDEX idx_reviews_rating ON Reviews(rating);
706 /*GROUP BY region in customer engagement query*/
707 • CREATE INDEX idx_customers_region ON Customers(region);
708 /*GROUP BY month in rating trend query*/
709 • CREATE INDEX idx_reviews_fulldate ON Reviews(Full_Date);
710 • SHOW INDEX FROM Reviews;

```

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
▶	reviews	0	PRIMARY	1	reviewid	A	10				BTREE			YES	
	reviews	0	uq_customer_product_review	1	customerid	A	9				BTREE			YES	
	reviews	0	uq_customer_product_review	2	productid	A	10				BTREE			YES	
	reviews	1	idx_reviews_rating	1	rating	A	3				BTREE			YES	
	reviews	1	idx_reviews_date	1	Full_Date	A	10				BTREE			YES	
	reviews	1	idx_reviews_productid	1	productid	A	9				BTREE			YES	
	reviews	1	idx_reviews_fulldate	1	Full_Date	A	10				BTREE			YES	

This dataset is small enough that the optimizer may correctly ignore your index. If that happens, say so and explain what changes at scale. An honest "no measurable improvement" scores better than an invented speedup.

5. **Write validation queries** that return a clear pass or fail: rows staged equals rows loaded plus rejected; no record loaded twice; all ratings in range; no orphaned foreign keys; all dates plausible; the combined view row count matches the review table; aggregate totals reconcile to the underlying rows.

```

808 /*5. Write validation queries that return a clear pass or fail: rows staged equals rows loaded plus rejected;
809 no record loaded twice; all ratings in range; no orphaned foreign keys; all dates plausible;
810 the combined view row count matches the review table; aggregate totals reconcile to the underlying rows.*/
811 /*rows staged equals rows loaded plus rejected*/
812 • SELECT
813     (SELECT COUNT(*) FROM Customer_survey) + (SELECT COUNT(*) FROM external_reviews_native) + (SELECT COUNT(*) FROM web_feedback_from_json)
814     (SELECT COUNT(*) FROM fact_review) AS total_loaded, (SELECT COUNT(*) FROM reject_log) AS total_rejected,
815     CASE
816         WHEN (SELECT COUNT(*) FROM Customer_survey)
817             + (SELECT COUNT(*) FROM external_reviews_native)
818             + (SELECT COUNT(*) FROM web_feedback_from_json)
819             = (SELECT COUNT(*) FROM fact_review)
820         THEN 'PASS'
821         ELSE 'FAIL'

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
total_staged	total_loaded	total_rejected	reconciliation_check
132	132	0	PASS

```

823 /*no record loaded twice*/
824 • SELECT 'Check 2: No Duplicates' AS check_name,
825     COUNT(*) AS duplicate_groups,
826     CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS result
827 FROM (
828     SELECT source_system, source_id
829     FROM fact_review
830     GROUP BY source_system, source_id
831     HAVING COUNT(*) > 1
832 ) dupes;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
check_name	duplicate_groups	result	
Check 2: No Duplicates	0	PASS	

```

833 /*all ratings in range*/
834 • SELECT 'Check 3: Ratings In Range' AS check_name,
835     COUNT(*) AS out_of_range_count,
836     CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS result
837 FROM fact_review
838 WHERE score_normalized IS NOT NULL
839 AND score_normalized NOT BETWEEN 0 AND 5;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
check_name	out_of_range_count	result	
Check 3: Ratings In Range	0	PASS	

```

841 • SELECT 'Check 4: No Orphaned FKs' AS check_name,
842        COUNT(*) AS orphan_count,
843        CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS result
844 FROM fact_review f
845 LEFT JOIN dim_customer c ON f.customer_id = c.customer_id
846 WHERE f.customer_id IS NOT NULL
847        AND c.customer_id IS NULL;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	check_name	orphan_count	result
▶	Check 4: No Orphaned FKs	0	PASS

```

847 /* all dates plausible;*/
848 • SELECT 'Check 5: Plausible Dates' AS check_name,
849        COUNT(*) AS implausible_count,
850        CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS result
851 FROM fact_review
852 WHERE review_date IS NOT NULL
853        AND (review_date > NOW() OR review_date < '2020-01-01');
854 -- the combined view row count matches the review table;
855 -- aggregate totals reconcile to the underlying rows.*/

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	check_name	implausible_count	result
▶	Check 5: Plausible Dates	0	PASS

```

371 /*the combined view row count matches the review table*/
372 • SELECT 'Check 6: Combined View Matches fact_review' AS check_name,
373        (SELECT COUNT(*) FROM vw_all_reviews_combined) AS combined_view_count,
374        (SELECT COUNT(*) FROM fact_review) AS fact_review_count,
375        CASE
376        WHEN (SELECT COUNT(*) FROM vw_all_reviews_combined) = (SELECT COUNT(*) FROM fact_review)
377        THEN 'PASS' ELSE 'FAIL'
378        END AS result;
379 /*aggregate totals reconcile to the underlying rows.*/
380 • SELECT 'Check 7: Aggregate Reconciliation' AS check_name,
381        fr_summary.source_system,

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	check_name	combined_view_count	fact_review_count	result
▶	Check 6: Combined View Match...	132	132	PASS

```

869  /*aggregate totals reconcile to the underlying rows.*/
870 • SELECT 'Check 7: Aggregate Reconciliation' AS check_name,
871         fr_summary.source_system,
872         fr_summary.reported_avg,
873         recompute.actual_avg,
874         CASE WHEN fr_summary.reported_avg = recompute.actual_avg THEN 'PASS' ELSE 'FAIL' END AS resu
875 FROM (
876     SELECT source_system, ROUND(AVG(score_normalized), 4) AS reported_avg

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	check_name	source_system	reported_avg	actual_avg	result
▶	Check 7: Aggregate Reconciliati...	customer_survey	3.4754	3.4754	PASS
	Check 7: Aggregate Reconciliati...	external_reviews	3.4697	3.4697	PASS

6. *Optional, for bonus marks:* a stored procedure that promotes staging to final tables inside a transaction with an error handler, or a stored function that centralizes a definition such as a rating band.

Deliverables

Submit one archive named `lastname_firstname_finalproject.zip` containing:

1. **ER diagram** (PNG or PDF) showing all tables, keys and cardinality.
2. **SQL scripts**, numbered so that running them in filename order rebuilds your database from nothing.
3. **Parsing scripts** for CSV, JSON and XML, commented and runnable from the command line.
4. **Report** (PDF or Word, 1,200–2,000 words plus tables) covering: design decisions and why; a data quality log with row counts per defect; your XML justification from Part 2.4; index strategy with **EXPLAIN** output; validation results; findings for the three objectives; and a **limitations** section stating what your analysis cannot support.
5. **README** – how to run everything, in order.